Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

ЗВІТ
з лабораторної роботи №1
з дисципліни «Програмування мовою Dart»
на тему: «Функції та обробка колекцій»
Варіант 3. Журнал особистих витрат

Виконав:
ст. гр. ПЗПІ-23-10
Вдовенко О.Д.

Перевірив:
асист. каф. ПІ
Баличев М.Д.

Харків 2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

Мета роботи: отримання навичок оголошення функцій із різними видами параметрів, побудови конвеєрів обробки колекцій засобами Iterable, а також обробки рядків, дат і регулярних виразів мовою Dart.

Завдання: розробити консольну програму, яка розбирає, перевіряє та агрегує набір записів предметної області за варіантом. Класи в роботі не використовуються: дані подаються колекціями та записами, обробка виконується функціями.

Варіант 3 — «Журнал особистих витрат». Основна сутність — витрата з полями: дата, категорія, сума, валюта, спосіб оплати, примітка.

Відповідно до порядку виконання роботи програма має:

- містити константний список вхідних рядків (не менше 15 записів, із них щонайменше 3 пошкоджені);
- розбирати рядок без аварійного завершення програми на пошкоджених даних;
- перевіряти діапазони значень і повертати перелік порушень;
- містити щонайменше три конвеєри обробки: фільтрацію, перетворення й згортання;
- групувати записи в `Map<String, List<…>>` і обчислювати агреговане значення для кожної групи;
- сортувати результат за двома критеріями з перемиканням напряму іменованим параметром;
- формувати звіт функцією з іменованими параметрами за допомогою `StringBuffer`;
- використовувати замикання, регулярний вираз з іменованими групами, `replaceAllMapped` і арифметику дат.

Повний код програми розміщено в репозиторії https://github.com/NureVdovenkoOleh/expense-journal (мітка `lab1`).

2 ОПИС ПРИЙНЯТИХ РІШЕНЬ

2.1 Структура програми

Проєкт створено командою `dart create -t console expense_journal`. Точка входу в каталозі `bin/` лише викликає функції бібліотеки й друкує результат; уся логіка винесена в `lib/`. Такий розподіл дозволяє імпортувати логіку в модульні тести та повторно використати її в наступних лабораторних роботах, де змінюватиметься насамперед реалізація, а не точка входу.

Бібліотеку поділено на файли за етапами обробки (табл. 2.1). Залежності між ними спрямовані в один бік: дані → розбір → перевірка → аналітика → звіт. Файл `lib/expense_journal.dart` лише реекспортує решту файлів, тому в `bin/` і тестах достатньо одного імпорту.

Таблиця 2.1 – Файли бібліотеки та їхнє призначення

Файл	Призначення
`expense.dart`	Тип запису `Expense`
`expense_data.dart`	Вхідні дані — константний список рядків
`expense_parser.dart`	Розбір рядка на витрату або опис помилки
`expense_validator.dart`	Перевірка діапазонів і допустимих значень
`expense_analytics.dart`	Фільтрація, згортання, групування, сортування
`expense_dates.dart`	Форматування дат, вік запису, замикання
`expense_report.dart`	Текстовий звіт і маскування номерів карток

2.2 Подання даних

Оскільки класи в роботі не використовуються, витрату подано записом (record) з іменованими полями (лістинг 2.1). На відміну від `Map<String, dynamic>`, запис має фіксований набір полів, а їхні типи перевіряє компілятор: звернення до неіснуючого поля або присвоєння значення іншого типу не скомпілюється. Суму подано типом `double`, оскільки вхідні суми містять копійки; похибка округлення двійкових дробів на результат не впливає, бо суми виводяться з двома знаками після коми.

Лістинг 2.1 – Тип запису витрати

```
typedef Expense = ({
  DateTime date,
  String category,
  double amount,
  String currency,
  String paymentMethod,
  String note,
});
```

Вхідні дані містять 21 рядок у форматі `дата;категорія;сума;валюта;спосіб оплати;примітка`. Роздільником

обрано крапку з комою, бо кома трапляється в примітках. Перші 15 рядків коректні; далі навмисно додано 4 пошкоджені рядки (пропущене поле, сума словами, дата у форматі `31.10.2026`, неіснуюча дата `2026-02-30`) і 2 рядки з допустимим форматом, але недопустимими значеннями (від’ємна сума, валюта `XYZ`).

2.3 Розбір рядка

Функція `parseExpense` (лістинг 2.2) ділить рядок методом `split`, перевіряє кількість полів і перетворює суму через `double.tryParse`. Для очікуваної помилки розбору не використовується `try/catch`: пошкоджений рядок — штатна ситуація, а не аварія. Функція повертає запис із двох полів — витрату та опис помилки, — тож код, який викликає функцію, бачить причину, з якої рядок відкинуто. Поля рядка розкладаються на змінні шаблоном списку.

Лістинг 2.2 – Функція розбору рядка

```
ParseResult parseExpense(String line) {
  final fields = line.split(fieldSeparator).map((f) => f.trim()).toList();
  if (fields.length != expectedFieldCount) {
    return _failure(
      'очікувалося $expectedFieldCount полів, а отримано ${fields.length}',
    );
  }

  final [dateText, category, amountText, currency, paymentMethod, note] =
      fields;

  final date = _parseDate(dateText);
  if (date == null) {
    return _failure('некоректна дата «$dateText»');
  }

  final amount = double.tryParse(amountText);
  if (amount == null) {
    return _failure('сума «$amountText» не є числом');
  }

  return (
    expense: (
      date: date,
      category: category,
      amount: amount,
      currency: currency,
      paymentMethod: paymentMethod,
      note: note,
    ),
    error: null,
  );
}
```

2.4 Перевірка дати регулярним виразом

`DateTime.tryParse` не відкидає неіснуючі дати, а переносить їх: рядок `2026-02-30` перетворюється на 2 березня. Тому функція `_parseDate` (лістинг 2.3) спершу

перевіряє формат регулярним виразом з іменованими групами `year`, `month`, `day`, а потім порівнює місяць і день отриманої дати з текстом. Шаблон записано сирим рядком `r'…'`, щоб зворотні скісні риски не потребували екранування.

Лістинг 2.3 – Перевірка формату та існування дати

```
final _datePattern = RegExp(r'^(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})
$');

DateTime? _parseDate(String text) {
  final match = _datePattern.firstMatch(text);
  if (match == null) {
    return null;
  }

  final date = DateTime.tryParse(text);
  if (date == null) {
    return null;
  }

  // DateTime.tryParse не відкидає неіснуючі дати, а переносить їх:
  // «2026-02-30» перетворюється на 2 березня. Тому порівнюємо місяць
  // і день отриманої дати з тим, що було записано в тексті.
  final isExistingDay =
      match.namedGroup('month') == _twoDigits(date.month) &&
      match.namedGroup('day') == _twoDigits(date.day);
  return isExistingDay ? date : null;
}
```

2.5 Валідатор

Функція `validateExpense` (лістинг 2.4) повертає перелік порушень; порожній список означає коректну витрату. Список будується елементами `if` усередині літерала колекції, без циклу й накопичувальної змінної. Допустимі валюти й способи оплати, межі суми та року винесено в іменовані константи.

Поточну дату не можна задати значенням параметра за замовчуванням (`{DateTime today = DateTime.now()}`), бо таке значення має бути константою. Тому параметр `today` оголошено nullable, а всередині функції використано вираз `today ?? DateTime.now()`. Це також дозволяє передавати фіксовану дату в тестах.

Лістинг 2.4 – Функція-валідатор

```
List<String> validateExpense(Expense expense, {DateTime? today}) {
  final reference = today ?? DateTime.now();
  return [
    if (expense.category.isEmpty) 'не вказано категорію',
    if (expense.amount <= 0) 'сума ${expense.amount} не є додатною',
    if (expense.amount > maxAmount)
      'сума ${expense.amount} перевищує $maxAmount',
    if (!supportedCurrencies.contains(expense.currency))
      'невідома валюта «${expense.currency}»',
    if (!supportedPaymentMethods.contains(expense.paymentMethod))
      'невідомий спосіб оплати «${expense.paymentMethod}»',
    if (expense.date.year < minYear) 'дата раніша за $minYear рік',
    if (expense.date.isAfter(reference)) 'дата витрати ще не настала',
  ];
}
```

2.6 Конвеєри обробки

Агрегація виконується ланцюжками `where` → `map` → `fold` (лістинг 2.5). Конвеєр фільтрації `inCurrency` відбирає витрати однієї валюти, оскільки суми в різних валютах додавати не можна. Конвеєр перетворення `categoriesOf` перетворює список витрат на множину назв категорій. Конвеєри згортання обчислюють загальну, середню й найбільшу суму.

У `largestExpense` використано `fold` з початковим значенням `null`, а не `reduce`: `reduce` бере перший елемент як початковий і на порожній колекції кидає виняток, тоді як `fold` просто повертає задане початкове значення.

Лістинг 2.5 – Конвеєри згортання

```
double totalAmount(Iterable<Expense> expenses, {required String currency})
=>
    inCurrency(expenses, currency: currency)
        .map((expense) => expense.amount)
        .fold(0.0, (total, amount) => total + amount);

Expense? largestExpense(
  Iterable<Expense> expenses, {
  required String currency,
}) => inCurrency(expenses, currency: currency).fold<Expense?>(
  null,
  (largest, expense) =>
      largest == null || expense.amount > largest.amount ? expense :
largest,
);
```

2.7 Групування та сортування

Функція `groupByCategory` (лістинг 2.6) згортає витрати в `Map<String, List<Expense>>`, а `totalsByCategory` обчислює суму для кожної групи. Функція `sortExpenses` приймає критерій сортування як значення типу `Comparator<Expense>` — функції `compareByDate` або `compareByAmount` — і перемикає напрям іменованим параметром `ascending`. Значенням за замовчуванням

параметра `compare` є функція верхнього рівня, яка є константою часу компіляції. Вихідна колекція не змінюється: сортується її копія.

Лістинг 2.6 – Групування та сортування

```
Map<String, List<Expense>> groupByCategory(Iterable<Expense> expenses) =>
    expenses.fold(<String, List<Expense>>{}, (groups, expense) {
      groups.putIfAbsent(expense.category, () => []).add(expense);
      return groups;
    });

List<Expense> sortExpenses(
  Iterable<Expense> expenses, {
  Comparator<Expense> compare = compareByDate,
  bool ascending = true,
}) {
  final Comparator<Expense> ordered = ascending
      ? compare
      : (a, b) => compare(b, a);
  return expenses.toList()..sort(ordered);
}
```

2.8 Замикання

Функція `olderThan` (лістинг 2.7) повертає функцію-предикат, яка перевіряє, чи витрата старша за заданий вік. Повернена функція є замиканням: вона захоплює параметр `age` і локальну змінну `reference` — момент, відносно якого рахується вік. Після завершення виклику `olderThan` ці змінні не зникають, а живуть стільки, скільки живе створений предикат. У звіті предикат передається методу `where`, щоб порахувати записи, давніші за 30 днів.

Лістинг 2.7 – Фабрика предикатів і її використання

```
bool Function(Expense) olderThan(Duration age, {DateTime? today}) {
  final reference = today ?? DateTime.now();
  return (expense) => expense.date.add(age).isBefore(reference);
}

  final isStale = olderThan(staleAge, today: reference);
```

2.9 Маскування номерів карток

Примітки можуть містити номери банківських карток. Функція `maskCardNumbers` (лістинг 2.8) знаходить їх регулярним виразом з іменованими групами `first` і `last` і методом `replaceAllMapped` замінює середні вісім цифр зірочками. Параметр функції заміни має тип `Match`, у якого немає методу `namedGroup`; замість приведення `as` використано перевірку `is!`, після якої тип звужується до `RegExpMatch` автоматично.

Лістинг 2.8 – Маскування номерів карток

```
final _cardNumberPattern = RegExp(
  r'\b(?<first>\d{4})[ -]?\d{4}[ -]?\d{4}[ -]?(?<last>\d{4})\b',
);

String maskCardNumbers(String text) =>
    text.replaceAllMapped(_cardNumberPattern, (match) {
      // Сюди завжди потрапляє RegExpMatch, але параметр має тип Match,
      // у якого немає namedGroup. Перевірка `is!` звужує тип без
      // приведення `as`.
      if (match is! RegExpMatch) {
        return match.group(0) ?? '';
      }
      final first = match.namedGroup('first');
      final last = match.namedGroup('last');
      return '$first **** **** $last';
    });
```

2.10 Арифметика дат

Вік запису в днях обчислюється методом `DateTime.difference`, який повертає `Duration`, і властивістю `inDays` (лістинг 2.9). Перед обчисленням обидві дати переводяться в UTC без часу доби: інакше перехід на зимовий час наприкінці жовтня додає годину до різниці, і кількість днів може відрізнятися на одиницю. Поріг давності задано константою `Duration(days: 30)`.

Лістинг 2.9 – Обчислення віку запису

```
int ageInDays(Expense expense, {DateTime? today}) {
  final reference = _dateOnly(today ?? DateTime.now());
  final Duration age = reference.difference(_dateOnly(expense.date));
  return age.inDays;
}

DateTime _dateOnly(DateTime date) =>
    DateTime.utc(date.year, date.month, date.day);
```

2.11 Формування звіту

Функція `buildReport` (лістинг 2.10) має іменовані параметри зі значеннями за замовчуванням: `topCount`, `ascending`, `separator`, `currency`, тому виклик `buildReport(expenses)` працює без налаштувань. Текст збирається у `StringBuffer`, а не оператором `+=`: рядки в Dart незмінні, і кожне `+=` створювало б новий рядок із копією всього попереднього тексту.

Лістинг 2.10 – Початок функції формування звіту

```
String buildReport(
  List<Expense> items, {
  int topCount = 5,
  bool ascending = false,
  String separator = ' | ',
  String currency = defaultCurrency,
  DateTime? today,
}) {
  final reference = today ?? DateTime.now();
  final buffer = StringBuffer()
    ..writeln('ЗВІТ ЗА ЖУРНАЛОМ ВИТРАТ')
    ..writeln('Сформовано: ${formatDate(reference)}')
    ..writeln('Коректних записів: ${items.length}')
    ..writeln()
    ..writeln('Підсумки ($currency)')
    ..writeln('  Разом: ${_money(totalAmount(items, currency: currency))}');
```

2.12 Залежності та налаштування аналізатора

Залежностей часу виконання програма не має — використано лише `dart:core`. Залежність `path`, яку додає `dart create`, прибрано, оскільки програма не працює з файлами. У розділі `dev_dependencies` залишено пакет `lints` (набір правил `recommended`, обов’язковий за вимогами дисципліни) і пакет `test` для модульних тестів. Як альтернативу `lints` розглянуто `very_good_analysis`, але він містить багато правил стилю понад вимоги дисципліни.

У файлі `analysis_options.yaml` до набору `recommended` додано режими `strict-casts`, `strict-inference`, `strict-raw-types` і правила `public_member_api_docs`, `prefer_final_locals`, `prefer_single_quotes`, `directives_ordering`. У коді немає жодного вживання оператора `!` (null assertion), ключового слова `late` і типу `dynamic`. Докладне обґрунтування рішень наведено у файлі README.md репозиторію.

3 РЕЗУЛЬТАТИ ВИКОНАННЯ

Результат роботи програми наведено на рис. 3.1. Із 21 вхідного рядка 4 пропущено під час розбору з поясненням причини, 2 не пройшли перевірку (від’ємна сума й невідома валюта), 15 витрат визнано коректними. Загальна сума витрат у гривнях — 6254.45, середня витрата — 521.20, найбільша — 1630.75 (категорія «Комунальні»). Номер картки в примітці замасковано.

```
> dart run
Building package executable...
Built expense_journal:expense_journal.
Вхідних рядків: 21
Рядок 16 пропущено: очікувалося 6 полів, а отримано 5
Рядок 17 пропущено: сума «сто двадцять» не є числом
Рядок 18 пропущено: некоректна дата «31.10.2026»
Рядок 19 пропущено: некоректна дата «2026-02-30»
Рядок 20 не пройшов перевірку: сума -150.0 не є додатною
Рядок 21 не пройшов перевірку: невідома валюта «XYZ»

ЗВІТ ЗА ЖУРНАЛОМ ВИТРАТ
Сформовано: 10.11.2026
Коректних записів: 15

Підсумки (UAH)
  Разом: 6254.45
  Середня витрата: 521.20
  Найбільша: Комунальні, 1630.75
  Записів, давніших за 30 днів: 10

За категоріями (UAH)
  Продукти: 2388.20
  Комунальні: 1630.75
  Навчання: 640.00
  Здоров’я: 410.00
  Кафе: 375.50
  Розваги: 350.00
  Зв’язок: 250.00
  Транспорт: 210.00

Найбільші витрати (UAH)
18.09.2026 | Комунальні | 1630.75 UAH | 53 дн. тому | Оплата за вересень
15.10.2026 | Продукти | 930.40 UAH | 26 дн. тому | АТБ
03.09.2026 | Продукти | 845.50 UAH | 68 дн. тому | Сільпо, закупівля на тиждень
30.10.2026 | Навчання | 640.00 UAH | 11 дн. тому | Друк методичних вказівок
27.09.2026 | Продукти | 612.30 UAH | 44 дн. тому | Ринок

Найновіші витрати
01.11.2026 | Розваги | 20.00 USD | 9 дн. тому | Гра в інтернет-магазині
30.10.2026 | Навчання | 640.00 UAH | 11 дн. тому | Друк методичних вказівок
24.10.2026 | Кафе | 155.50 UAH | 17 дн. тому | Обід, картка 5375 **** **** 9876
19.10.2026 | Здоров’я | 410.00 UAH | 22 дн. тому | Аптека
15.10.2026 | Продукти | 930.40 UAH | 26 дн. тому | АТБ
```

Рисунок 3.1 – Результат виконання команди dart run

Статичний аналізатор не виявив зауважень (рис. 3.2), усі 19 модульних тестів пройдено успішно (рис. 3.3). Команда `dart format` змін у коді не внесла.

```
PS E:\TRUE FORM OF HOMEWORK\голая грудь 4\сем 1\Dart\Lb1\practice\expense_journal> dart analyze
Analyzing expense_journal...
No issues found!
PS E:\TRUE FORM OF HOMEWORK\голая грудь 4\сем 1\Dart\Lb1\practice\expense_journal>
```

Рисунок 3.2 – Результат виконання команди dart analyze

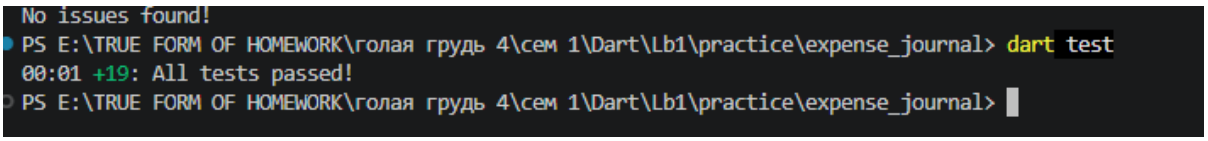


Рисунок 3.3 – Результат виконання команди dart test

Роботу виконано поетапно; історію змін у репозиторії наведено в табл. 3.1.

Таблиця 3.1 – Історія комітів у репозиторії

Дата	Зміст коміту
05.11.2026	Створено проєкт і вхідні дані журналу витрат
07.11.2026	Додано розбір рядків і валідатор витрат
08.11.2026	Додано конвеєри агрегації, групування й сортування
09.11.2026	Додано текстовий звіт, маскування карток і обчислення віку записів
10.11.2026	Додано обґрунтування проєктних рішень у README

ВИСНОВКИ

У даній лабораторній роботі було розроблено консольну програму розбору, перевірки та агрегації журналу особистих витрат; засвоєно оголошення функцій із позиційними та іменованими параметрами, побудову конвеєрів обробки колекцій, роботу з рядками, датами й регулярними виразами мовою Dart.

Подання даних записами замість класів дало типобезпечну структуру без додаткового коду, але показало й обмеження: поведінку, пов’язану з витратою (розбір, перевірка, форматування), довелося розмістити в окремих функціях. У лабораторній роботі 2 ці функції природно стануть конструкторами й методами класу.

Використання `tryParse` і запису «результат — помилка» дозволило обробити всі пошкоджені рядки без винятків. Окремо виявлено, що `DateTime.tryParse` не відкидає неіснуючі дати, тому для перевірки дати знадобився регулярний вираз з іменованими групами. Конвеєри `where → map → fold` замінили цикли з накопичувальними змінними, а використання `fold` замість `reduce` зробило агрегацію безпечною для порожніх колекцій.

ПЕРЕЛІК ДЖЕРЕЛ

1. Dart Team. Functions. Dart documentation. URL: https://dart.dev/language/functions (date of access: 10.11.2026).
2. Dart Team. Records. Dart documentation. URL: https://dart.dev/language/records (date of access: 10.11.2026).
3. Dart Team. Patterns. Dart documentation. URL: https://dart.dev/language/patterns (date of access: 10.11.2026).

4. Dart Team. Collections. Dart documentation. URL: https://dart.dev/language/collections (date of access: 10.11.2026).
5. Dart Team. Effective Dart. URL: https://dart.dev/effective-dart (date of access: 10.11.2026).
6. Dart Team. Iterable class — dart:core library. URL: https://api.dart.dev/stable/dart-core/Iterable-class.html (date of access: 10.11.2026).
7. Dart Team. DateTime class — dart:core library. URL: https://api.dart.dev/stable/dart-core/DateTime-class.html (date of access: 10.11.2026).
8. Dart Team. RegExp class — dart:core library. URL: https://api.dart.dev/stable/dart-core/RegExp-class.html (date of access: 10.11.2026).
9. Dart Team. StringBuffer class — dart:core library. URL: https://api.dart.dev/stable/dart-core/StringBuffer-class.html (date of access: 10.11.2026).
10. Dart Team. Linter rules. URL: https://dart.dev/tools/linter-rules (date of access: 10.11.2026).